LAB 3

COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Food Bank Surplus Redistribution Platform

NAME : Shweta Mailaragouda Patil

SRN : PES2UG25CS825

Objective

Evaluate different architectural styles, select the most appropriate one for the assigned scenario, and create a comprehensive UML Component Diagram showing modules, interfaces, and dependencies.

Learning Outcomes Addressed

- Compare and contrast Layered, Microservices, and Client-Server architectural styles.
- Analyse the advantages and disadvantages of each style relative to the assigned scenario.
- Choose and justify the most suitable architectural style based on system requirements.
- Create a UML Component Diagram using proper component and interface notation.
- Show provided and required interfaces and component dependencies.
- Document the architectural decision and explain security and performance implications.

Step 1: Scenario Review

Food Bank Surplus Redistribution Platform

The Food Bank Surplus Redistribution Platform is designed to help manage the collection and redistribution of surplus food to people or organizations who need it. The system connects food donors, food bank administrators, and recipients through a centralized platform.

The system allows donors to register surplus food by providing details such as the type of food, quantity, availability, and relevant collection information. The platform maintains these food donation records so that available surplus food can be identified and redistributed efficiently.

The system also supports food reservation, allowing eligible recipients or authorized users to request available food items. The platform must check the availability of the requested food before confirming a reservation and should prevent the same available quantity from being allocated to multiple requests.

Once a food item has been reserved, the system needs to manage the order and reservation information. The Order Manager Component is responsible for coordinating these activities and communicating with other components involved in the process.

The system also includes a Payment Service Component, which is one of the components specified in the Lab 3 scenario. This component can handle payment-related requests when a transaction requires payment processing.

Another important part of the system is pickup scheduling. After a reservation or order is confirmed, the platform should support the arrangement of a suitable pickup time and maintain the required pickup information.

The platform should maintain accurate food availability information throughout the redistribution process. When food is reserved, collected, or becomes unavailable, the corresponding information should be updated so that users receive correct availability details.

The system should provide clear communication between its components so that food donation, reservation, order, payment, and pickup activities can work together without unnecessary duplication of information.

Key Functional Requirements

- Post Surplus Food – donors provide surplus-food information such as quantity, expiry time, and dietary information.
- View Available Food – shelters view available food batches.
- Reserve Food Batch – a shelter reserves an available batch.
- Schedule Pickup – a pickup window is selected for a reserved batch.
- Verify Pickup – pickup is verified using the information generated for the reservation.
- Send Notifications – relevant users receive system notifications.
- Check Food Availability – the system checks whether a food batch is still available.
- Handle Expired Batch – expired food batches are deactivated so they are no longer treated as available.

Key Non-Functional Requirements

- Performance: operations involving availability, reservation, pickup, and notifications should be responsive.
- Usability: the system should provide clear interactions for donors and verified shelters.
- Security: user access and protected operations should be controlled through authentication/authorization.

Main Challenges

Performance: The system should process food availability, reservation, order, and pickup requests efficiently.

Usability: Users should be able to donate, reserve, and manage food-related activities through a simple and understandable system.

Security: User information and transaction-related information should be protected from unauthorized access.

Data Consistency: Food quantities and availability must remain accurate when multiple reservations or updates occur.

Component Communication: The different components must communicate correctly through well-defined interfaces.

Maintainability: Changes to one functionality should have minimal impact on the other components.

Scalability: The system should be capable of handling an increasing number of food donations, reservations, and users.

Step 2: Architectural Style Analysis

Style	Structure	Pros	Cons
Layered Architecture	Horizontal layers such as Presentation, Business, and Data.	Clear separation of concerns; easy to understand; good for traditional applications.	Performance overhead; tight coupling between layers; difficult to scale individual components.
Microservices Architecture	Small, independently deployable services.	Independent scaling; fault isolation; technology diversity; independent deployment.	Operational complexity; network latency; data consistency challenges.
Client-Server Architecture	Centralized server with multiple clients.	Centralized control; simple deployment; easy data consistency.	Single point of failure; scalability bottleneck; limited fault tolerance.

Scenario-Based Analysis

Layered Architecture fits a traditional platform in which user-facing functions, business rules, and stored information have clear responsibilities. Microservices can be useful when independent scaling and deployment are important, but they introduce the operational and consistency concerns identified in the handout. Client-Server is simple and centralized, but its single-server nature can create a bottleneck or single point of failure.

Architecture Decision Matrix

Criterion	Layered	Microservices	Client-Server	Preferred
Separation of concerns	High	High	Medium	Layered
Implementation simplicity	High	Low	High	Layered
Independent scaling	Low	High	Low	Microservices
Operational complexity	Low	High	Low	Layered
Data consistency simplicity	High	Medium/Low	High	Layered

Step 3: Architecture Selection & Component Identification

Selected Architecture

We chose Layered Architecture for the Food Bank Surplus Redistribution Platform.

The selection is based on the system's clear separation of presentation, business processing, and data responsibilities, together with the need for a straightforward architecture for the specified scenario.

Component Identification

No.	Component	Role in the Assigned System
1	Order Manager Component	Coordinates order/reservation-related processing and interactions between system functions.
2	Payment Service Component	The second component explicitly given in the Lab 3 instructions; represents payment-processing service interaction.
3	Food Donation Management Component	Manages posting and handling of surplus-food information and expiry-related processing.
4	Food Reservation Component	Checks availability and manages reservation of an available food batch.
5	Pickup Scheduling Component	Manages pickup-window selection and pickup verification/status.

Component Responsibilities in More Detail

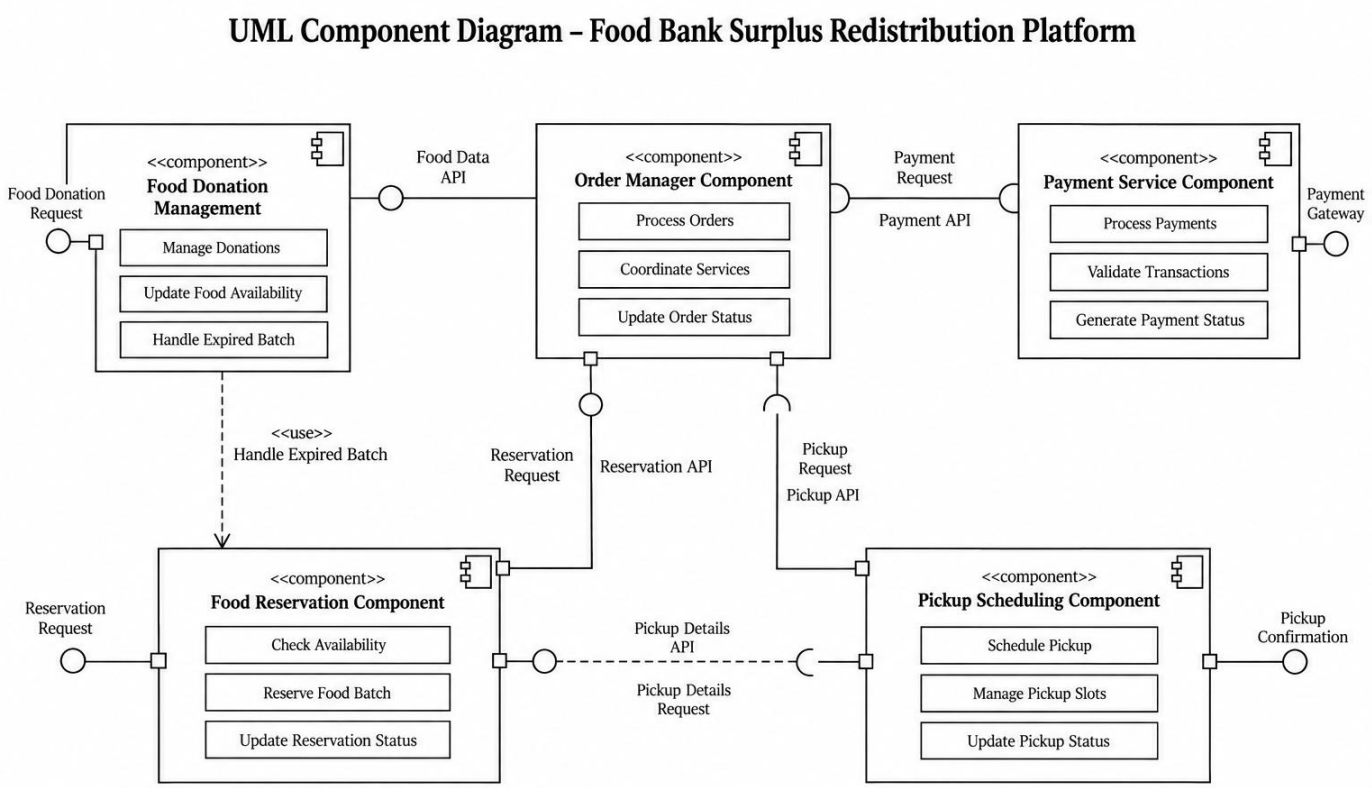
- Order Manager – receives and coordinates relevant requests and communicates with payment and reservation functions.
- Payment Service – exposes the payment-processing service required by the exercise.
- Food Donation Management – manages food-batch details, including quantity, expiry and dietary information.
- Food Reservation – checks availability and reserves a batch for a shelter.
- Pickup Scheduling – manages the pickup window and verification-related information.

Step 4: Component Diagram Creation

Setup and Arrangement

- UML tool: draw.io, as recommended by the handout.
- Components are represented using rectangles with the «component» stereotype and component icon.
- The five components are arranged logically so related application functions are grouped together.
- Interfaces are attached to component interactions and labelled with API/request terminology.

UML Component Diagram



Interface Requirements

The diagram contains more than the required four interfaces and shows the required Order Manager ↔ Payment Service interaction.

No.	Components	Interface	Data Flow	Notation
1	Food Donation Management ↔ Order Manager	Food API	Food details: quantity, expiry time, dietary information	Provided / Required
2	Order Manager ↔ Payment Service	Payment API	Payment processing request	Provided / Required
3	Order Manager ↔ Food Reservation	Order API	Reservation/order information and availability request	Provided / Required
4	Food Reservation ↔ Pickup Scheduling	Reservation/Pickup API	Reserved batch, pickup window and verification details	Provided / Required
5	Food Reservation ↔ Order Manager	Availability API	Availability and reservation status	Provided / Required

Data Flow and Component Interactions

1. Food Donation Management sends food details to the Order Manager.

2. The Order Manager coordinates processing and sends the payment-processing request to the Payment Service interface shown in the lab instructions.

3. The Order Manager sends reservation/order information to Food Reservation.

4. Food Reservation checks availability and returns availability/reservation status through the corresponding interface.

5. Food Reservation sends pickup information to Pickup Scheduling, including the selected pickup window and verification-related details.

Interface Notation

- Provided Interface: Represented by a ball (lollipop) symbol. It shows a service that a component provides to other components.
- Required Interface: Represented by a socket (semicircle) symbol. It shows a service that a component needs from another component.
- Interface Connection: Provided and required interfaces are connected to show how components communicate.
- Interface Labels: Interfaces should be clearly labelled according to the service provided, such as Payment Processing Request, Food Data, Reservation Request, or Pickup Request.
- Communication Lines: Use solid lines to represent internal service communication between components.
- Data Flow: The direction of requests and information exchanged between components should be clearly indicated.

Step 5: Written Justification

Architecture Selection: We chose Layered Architecture for the Food Bank Surplus Redistribution Platform.

Architectural Choice: Layered Architecture is selected because the Food Bank Surplus Redistribution Platform has clearly separated responsibilities. The system provides functions for posting surplus food, viewing available food, checking availability, reserving food batches, scheduling pickups, verifying pickups, sending notifications, and handling expired batches. These responsibilities can be organized into presentation, business, and data-related layers. Such separation gives the system a clear structure and makes the relationships between its components easier to understand.

Reason 1: The first reason for selecting Layered Architecture is separation of concerns. User interaction can be separated from the business rules that control food availability, reservation, pickup scheduling, and expiry. Data operations can be kept separate from the user-facing functions. This means that each part of the system has a defined responsibility instead of placing all functionality in one large module. The structure also makes it easier to locate and understand individual operations.

Reason 2: The second reason is simplicity and maintainability. The assigned Food Bank system is a traditional application scenario with a defined collection of functions. It does not require every function to be independently deployed. A layered structure is therefore easier to develop, test, debug, and maintain. If a business rule changes, such as the conditions for reserving a food batch, the relevant business component can be changed without redesigning the complete system.

Comparison with Microservices: Microservices architecture consists of small independently deployable services and can provide independent scaling, fault isolation, technology diversity, and independent deployment. However, the Lab 3 handout also identifies operational complexity, network latency, and data consistency challenges. For the current scope of the Food Bank platform, those additional concerns may make the solution unnecessarily complex.

Comparison with Client-Server: Client-Server architecture provides centralized control, simple deployment, and easy data consistency. However, the handout identifies a single point of failure, scalability bottleneck, and limited fault tolerance as disadvantages. Layered Architecture gives the application a clear internal structure while still supporting centralized processing.

Suitability for the Scenario: Layered Architecture is therefore a practical choice for the assigned system. The presentation responsibilities can support donors and verified shelters, business responsibilities can manage donation, reservation, and pickup rules, and data responsibilities can manage the information required by those operations. This organization is consistent with the component-modelling objective of separating system functionality into modular components.

Security Advantage: Security is important because the platform handles user accounts, verified shelters, food donations, reservations, pickup information, and verification information. Layered Architecture allows authentication and authorization checks to be applied before protected business operations are performed. The business layer can enforce rules such as restricting reservation operations to appropriate verified shelters or authorized users. Keeping these checks in defined responsibilities makes security rules easier to identify, maintain, and test.

Performance Benefit: Layered Architecture provides a practical performance benefit for this scenario because common operations can follow a predictable path through the application's business and data responsibilities. For example, checking food availability and processing a reservation can be handled without requiring communication among a large number of independently deployed services. This avoids some of the network latency associated with a distributed microservices design and provides an efficient structure for the current scope.

Component Interaction and Data Flow: The five-component model demonstrates how the main functions interact. Food Donation Management provides food information to the Order Manager. The Order Manager coordinates the relevant request and communicates with the Payment Service through the payment-processing interface specified in the Lab 3 instructions. The Order Manager communicates reservation information to Food Reservation. Food Reservation checks availability and passes reserved-batch and pickup-window information to Pickup Scheduling. These interactions show both component dependency and the information exchanged between components.

Usability Consideration: Usability is also considered in the architectural decision. Donors should be able to post surplus food with the required details, while shelter users should be able to find available food, reserve a batch, and schedule pickup without unnecessary complexity. Separating presentation responsibilities from business logic makes it easier to improve user interaction without changing all of the underlying business rules.

Overall Justification: Overall, Layered Architecture provides a clear and maintainable structure for the Food Bank Surplus Redistribution Platform. It supports the required functions, provides clear separation of responsibilities, allows security controls to be applied to protected operations, and provides a predictable processing path for common requests. Microservices offers useful independent scaling and fault isolation but introduces additional operational, network, and consistency complexity. Client-Server is simple and centralized but has the single-point-of-failure and scalability limitations described in the handout. Therefore, Layered Architecture is the most appropriate choice for this assigned scenario

Interface Notation and UML Requirements

- Provided Interface – represented using the UML ball/circle notation. It represents a service that a component offers to another component.
- Required Interface – represented using the UML socket/semicircle notation. It represents a service that a component needs from another component.
- Ports – interaction points between a component and its environment; related provided and required interfaces may be grouped at an interaction point.
- Assembly Connector – connects a provided interface to a required interface so the service relationship is explicit.
- Usage Dependency – shown using a dashed arrow with the «use» stereotype.
- Internal Service Communication – shown using solid lines as specified in the handout.
- Interface Labels – identify the service/protocol or interaction, such as API calls, requests, and database-related queries.
- Data Flow – labels identify the information exchanged, such as food details, payment requests, reservation information, availability status, and pickup details.

Conclusion

- The **Food Bank Surplus Redistribution Platform** requires a well-structured architecture to manage food donations, reservations, orders, payments, and pickup scheduling efficiently. The selected **Layered Architecture** provides clear separation of responsibilities and makes the system easier to develop, maintain, and modify.
- The UML Component Diagram represents the **five required components** and their interactions through clearly defined interfaces. The use of proper UML interface notation, dependencies, and data-flow connections helps demonstrate how the components communicate within the system. Overall, the proposed architecture provides a suitable

foundation for building a **secure, reliable, maintainable, and efficient** food redistribution platform.